

Reporte de Evaluación Parcial 2 - Felipe

Evaluación General

La revisión técnica de esta entrega demuestra una evolución técnica sobresaliente. Tu proyecto "Pokémon TCG" no solo cumple con los requisitos de una aplicación Full Stack, sino que exhibe un nivel de ingeniería y "Developer Experience" (DX) poco común para el segundo año de la carrera. La elección de FastAPI y SQLAlchemy demuestra ambición y ganas de construir herramientas de alto rendimiento. El sistema de despliegue automatizado (`EJECUTAR_SERVIDOR.bat` y `run.py`) es un detalle brillante que asegura que la aplicación funcione en cualquier entorno, lo que habla de un enfoque profesional hacia la distribución de software.

Fortalezas

- **Arquitectura del Backend y DX:** El uso de FastAPI junto con SQLAlchemy y Pydantic establece una arquitectura moderna y robusta. El script de inicialización que valida la versión de Python, maneja dependencias y levanta el servidor (`run.py`) es un excelente añadido que mejora la experiencia de desarrollo.
- **Modelado y Persistencia (DDL):** El archivo `models.py` muestra un modelado ORM avanzado. La implementación de la tabla intermedia `Favorite` para la relación muchos-a-muchos (Usuarios y Cartas) con sus respectivas `ForeignKey` y restricciones de unicidad (`UniqueConstraint`), garantiza la integridad referencial y protege la base de datos a nivel estructural.
- **Robustez y Validación (Defensa en Profundidad):** El uso de `schemas.py` con Pydantic para validar entradas (ej. `min_length` , validadores customizados) es una de las mejores defensas contra inyecciones y datos malformados. Además, la implementación de encriptación de contraseñas (`bcrypt`) demuestra conciencia sobre ciberseguridad.
- **Integración Full-Stack:** El frontend (Vanilla JS) consume eficientemente los endpoints RESTful documentados automáticamente por FastAPI. El manejo de estado y sesiones en `app.js` está bien estructurado.

Aspectos a Mejorar (Detalles que impidieron la nota máxima)

Tu trabajo es excepcional, pero en el mundo profesional, los detalles arquitectónicos y de seguridad en el despliegue son los que marcan la diferencia entre un prototipo brillante y un producto listo para producción. Por esto, no alcanzas el puntaje perfecto:

- **Estructura del Proyecto (Separación de Capas):** Si bien es destacable que el mini-juego de React esté correctamente aislado en su propia carpeta (`web_mini_game/`), el "core" del frontend (`index.html` , `app.js` , `styles.css` principales) donde se inyecta dicho juego sigue mezclado en la raíz del proyecto junto a los scripts de ejecución. Un principio básico de arquitectura es el aislamiento estricto de todas las capas. Todo el código de interfaz base debe vivir en una carpeta dedicada (ej. `/public` o `/client`) para no saturar la raíz del repositorio.
- **Gestión de CORS y Seguridad (Fricción vs Producción):** En el backend, el middleware de CORS está configurado para aceptar cualquier origen (`allow_origins=["*"]`). Si bien esto elimina la fricción durante el desarrollo local, es una vulnerabilidad severa en producción. Enseñar a cerrar esos permisos es vital en el eje de Ciberseguridad del curso.

Desglose por Rúbrica

- Modelado y Persistencia: 25/25
- Arquitectura de API: 25/25
- Frontend y UX: 19/20 (Descuento leve por la mezcla de archivos estáticos en la raíz)

- Integración Full-Stack: 15/15
- Robustez y Validación: 10/10
- Orden y Git: 4/5 *(Descuento por no aislar la capa Frontend en su propio directorio y dejar CORS abierto).*

Puntos Extra (+): Relaciones Técnicas avanzadas (Many-to-Many). Capa de seguridad sólida (Hashing de contraseñas, validación con Pydantic). Documentación viva mediante Swagger (FastAPI).

Resultado Final

- Puntaje: 98/100
- Nota: 6,9

Síntesis Técnica

Es un trabajo excelente que arriesga con un stack tecnológico desafiante (Python/FastAPI) y sale victorioso. La inclusión de un launcher personalizado denota una mentalidad orientada al producto y a la experiencia del usuario (incluso si ese usuario es otro desarrollador). Tienes una base muy fuerte; el siguiente paso es afinar el orden de tu repositorio para separar visualmente las capas de frontend y backend, y ajustar los permisos de seguridad (CORS) pensando siempre en un entorno de producción real.